IEORE4004 Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University
Department of Industrial Engineering and Operations Research

Classes of Mathematical Programs

Program Type	Decision Variables	Objective and Constraints
Linear Program	Continuous	Linear functions
Integer Program	Integer	Linear functions
Mixed Integer Linear Program	Both integer and continuous	Linear functions
Binary Program	Binary	Linear functions
Nonlinear Program	Continuous	Nonlinear functions

Linear Programming

- Solving a Linear Program Graphically
- The Simplex Method
- Duality Theory
- Post Optimality and Sensitivity Analysis

Solving a Linear Program Graphically

Step 1: Graph the region that satisfies all of the constraints.

Example:

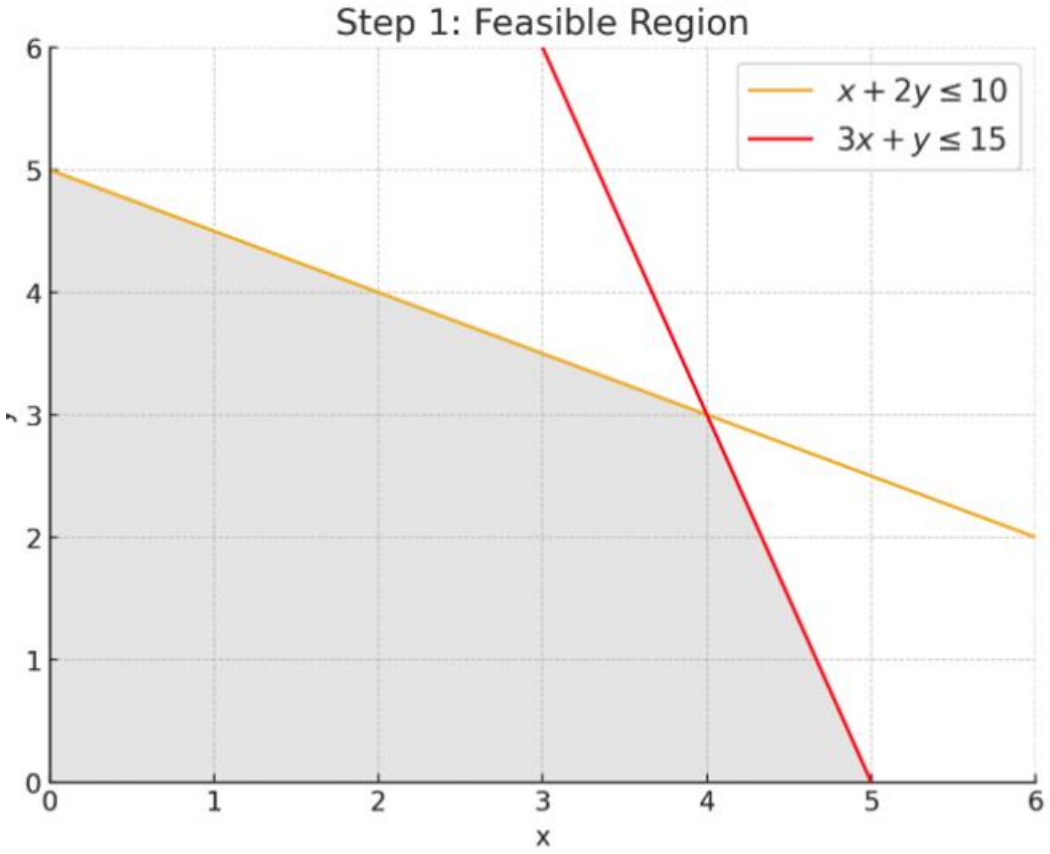
- Constraints:

  $x+2y \leq 10$

  $3x+y \leq 15$

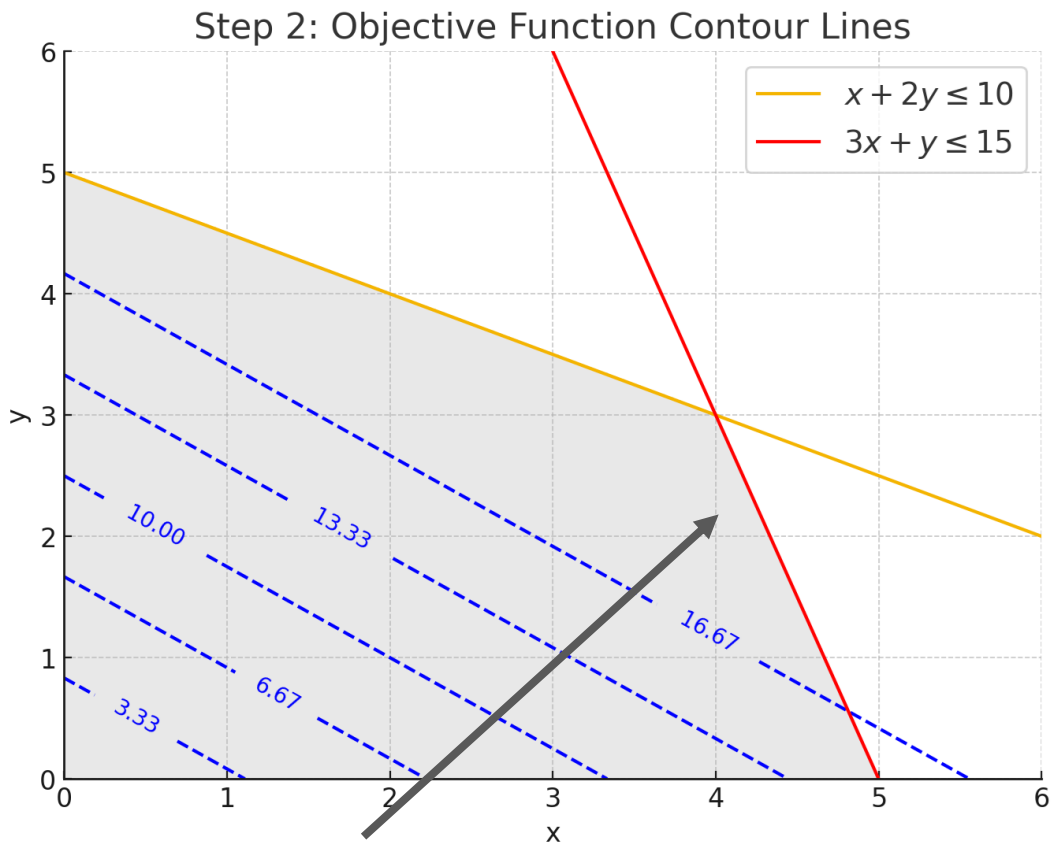
  $x \geq 0$

  $y \geq 0$



Solving a Linear Program Graphically

Step 2: Draw a contour line of the objective function and determine which direction improves the objective function. A second contour line can be used to determine which direction is improving.



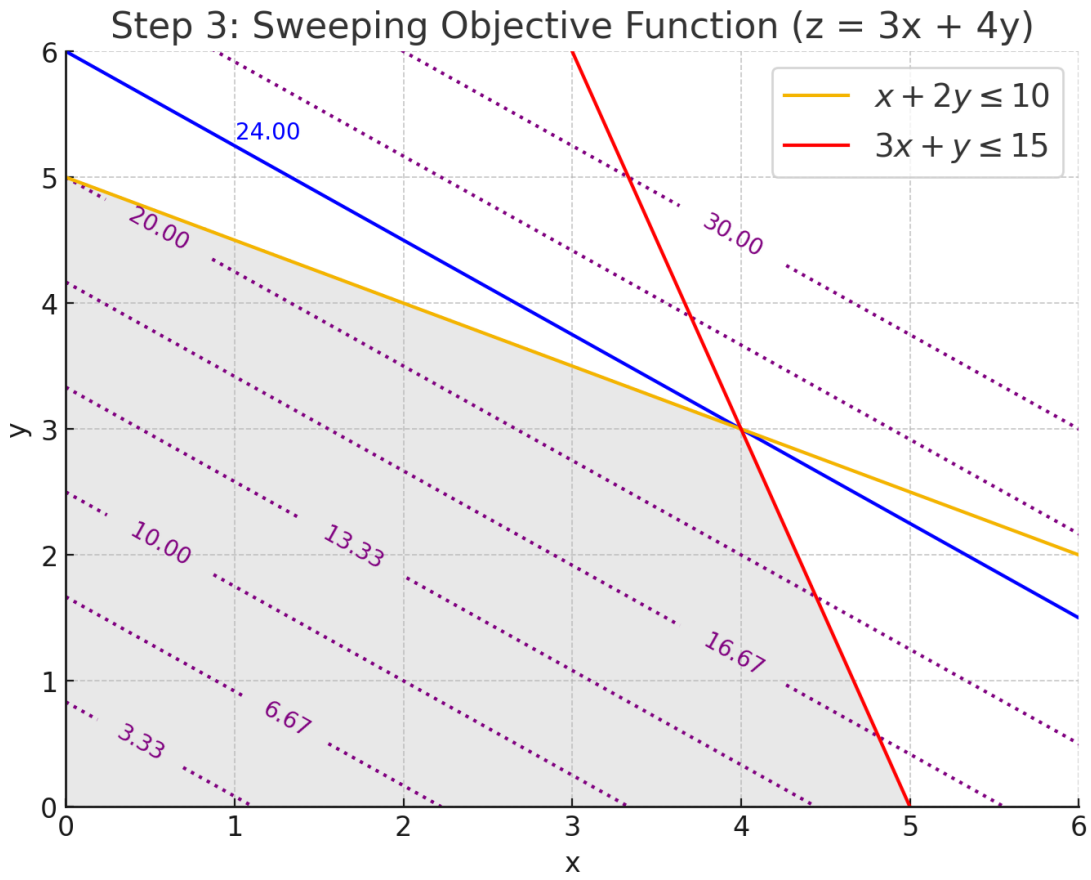
Objective function: z = 3x+4y

- The improving direction is towards higher values as we aim to maximize z

Solving a Linear Program Graphically

Step 3: Sweep the contour line in the improving direction, generating parallel lines, until it is about to leave the region. The last line intersecting the region has the optimal objective function value. The point(s) where this line intersects the region is optimal.

- Here, solid blue line represents z=24, which is the maximum value achievable within the feasible region.



Solving a Linear Program Graphically

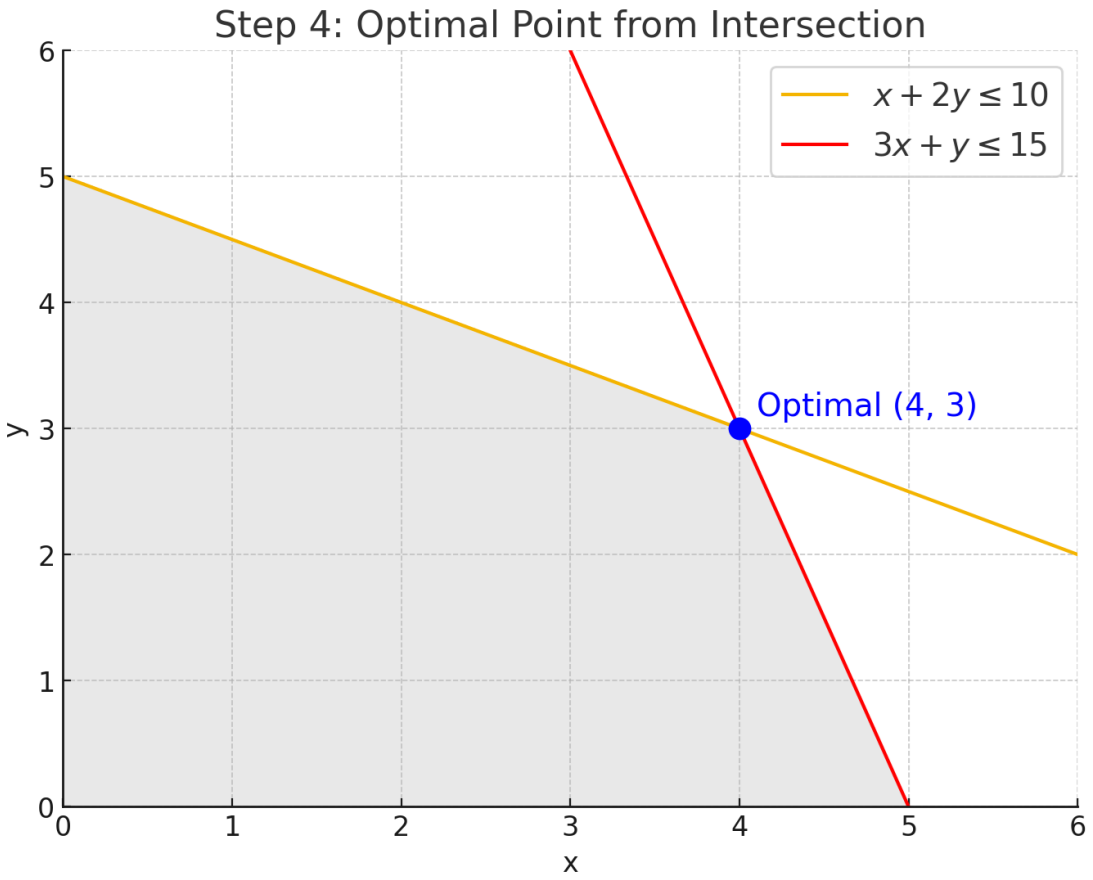
Step 4: Find the coordinates of an optimal point by solving a system of two linear equations from the constraints whose lines pass through this point.

- Solve the system:

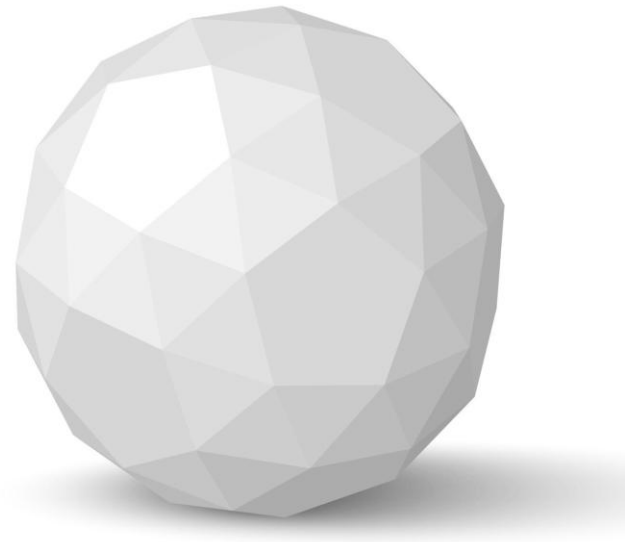
  x+2y=10

  3x+y=15

  Solution => (x,y)=(4,3)



The Simplex Method



The Simplex Method

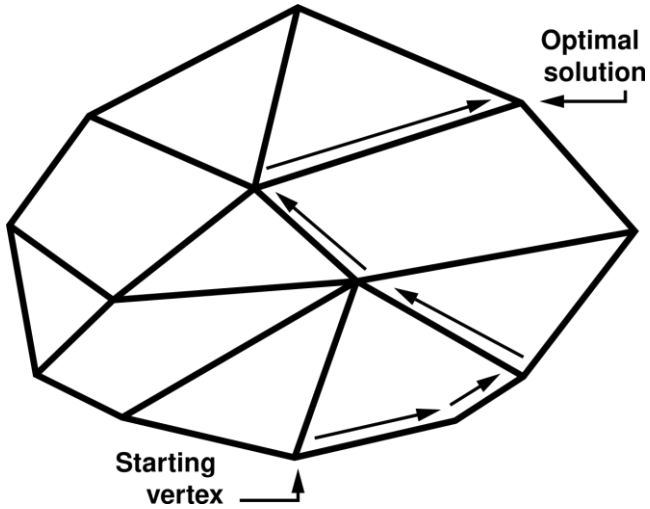
- The simplex method is a **powerful algorithm** used to solve linear programming problems

The Simplex Method

- The simplex method is a **powerful algorithm** used to solve linear programming problems
- Can be used to solve small LP problems **by hand**

The Simplex Method

- The simplex method is a **powerful algorithm** used to solve linear programming problems
- Can be used to solve small LP problems **by hand**
- The method **systematically moves along the edges of the feasible region**, which is defined by the constraints, to find the optimal solution.



The Simplex Method

- The simplex method is a **powerful algorithm** used to solve linear programming problems
- Can be used to solve small LP problems **by hand**
- The method **systematically moves along the edges of the feasible region**, which is defined by the constraints, to find the optimal solution.
- This **iterative process** continues until no further improvements are possible, resulting in the optimal solution.

Duality Theorems

(P)

$$\max \mathbf{c}^T\mathbf{x}$$
$$\mathbf{Ax} \leq \mathbf{b}$$
$$\mathbf{x} \geq 0$$

(D)

$$\min \mathbf{b}^T\mathbf{y}$$
$$\mathbf{A}^T\mathbf{y} \geq \mathbf{c}$$
$$\mathbf{y} \geq 0$$

Primal (Max)	Dual (Min)
i-th constraint $\leq$	variable $y_i \geq 0$
i-th constraint $\geq$	variable $y_i \leq 0$
i-th constraint $=$	variable y_i unrestricted
$x_i \geq 0$	i-th constraint $\geq$
$x_i \leq 0$	i-th constraint $\leq$
x_i unrestricted	i-th constraint $=$

Duality Theorems – Weak Duality

1. Weak Duality Theorem

! In a maximization problem if x is any feasible solution of the primal and y is any feasible solution of the dual, then

$$val(P) \leq val(D),$$

(can also be written as: $c^T x \leq b^T y$)

- In other words, the value of any feasible solution to the **dual** yields an **upper bound** on the value of any feasible solution (including the optimal) to the **primal**

Duality Theorems – Strong Duality

2. Strong Duality Theorem

! If x^* is an optimal solution to the primal and y^* is an optimal solution to the dual, then $val(P) = val(D)$
(can also be written as: $c^T x^* = b^T y^*$)

- In other words, if the **dual** yields to an optimal solution, the **primal** also will have the same optimal solution.

Duality Theorems – Complementary Slackness

3. Complementary Slackness Theorem

- At optimality, if a constraint is not tight (has a positive slack) in the primal, then the corresponding dual variable must be 0:

$$A_i x \neq b_i \rightarrow y_i = 0$$

- Conversely, if a primal variable is strictly positive, then the corresponding dual constraint must be tight (i.e., slack is 0):

$$x_i > 0 \rightarrow A_i^T y = c_i$$

Post Optimality and Sensitivity Analysis

- **Often the values of model parameters are just estimates from data**
- Sensitivity analysis investigates the impact changes in the parameters (a_{ij} , b_i, c_j) have on the optimal solution. There are three types of sensitivity analyses:
  - Identifying the sensitive parameters ***(e.g., when even a very small change in the value of parameters has some impact on the optimal solution)***
  - Determining the range of parameters within which the optimal solution remains unchanged
  - Re-optimization ***(e.g., finding the new optimal solution when there is a change in the parameters)***

Integer Programming

- Multiple Knapsack Problem
- Modeling Logical Conditions
- Branch and Bound Method
- Cutting Planes Algorithm

Multiple Knapsack Problem

Let m be the number of knapsacks, and W_i be the capacity of knapsack i.

- x_{ij} is 1 if item j is placed in knapsack i, and 0 otherwise

$$\text{Maximize} \quad \sum_{i=1}^{m} \sum_{j=1}^{n} v_j x_{ij}$$

$$\text{subject to} \quad \sum_{j=1}^{n} w_j x_{ij} \leq W_i \qquad \forall i = 1, 2, \ldots, m$$

$$\sum_{i=1}^{m} x_{ij} \leq 1 \qquad \forall j = 1, 2, \ldots, n$$

$$x_{ij} \in \{0, 1\} \qquad \forall i = 1, 2, \ldots, m \quad \forall j = 1, 2, \ldots, n$$

Modeling Logical Conditions

Question: If we want to restrict x to be one of the elements {2, 7, 12}, how should we model it?

$$x = 2w_1 + 7w_2 + 12w_3$$
$$w_1 + w_2 + w_3 = 1$$
$$w_i \in \{0,1\} \, for \, i = 1 \, to \, 3$$

Question: If we want to restrict x to be one of the elements {0, 2, 7, 12}, how should we model it?

$$x = 2w_1 + 7w_2 + 12w_3$$
$$w_1 + w_2 + w_3 \leq 1$$
$$w_i \in \{0,1\} \, for \, i = 1 \, to \, 3$$

Modeling Logical Conditions

Question: How can we restrict x to be either 0 or between particular positive bounds?

In algebraic notation: $$x = 0 \text{ or } l \leq x \leq u$$

Define:

$$y = \begin{cases} 0, & for\ x = 0 \\ 1, & for\ l \leq x \leq u \end{cases}$$

Then, the logical constraints can be modeled by:

$$ly \leq x \leq uy, \qquad where \quad y \in \{0,1\}$$

Modeling Logical Conditions

Question: If we want to have either one of these constraints:

$2x_1 + 3x_2 \geq 14$ **or** $5x_2 - 7x_3 \leq 3$ how should we model it?

$$2x_1 + 3x_2 \geq 14 - M(1 - w)$$
$$5x_2 - 7x_3 \leq 3 + Mw$$
$$w \in \{0,1\}$$

Branch and Bound Algorithm

Branch and Bound Algorithm

Let's consider this IP to explain the method:

$$\textit{Maximize } Z = 5x_1 + 6x_2$$

$$\textit{Subject to} \quad x_1 + x_2 \leq 5$$
$$4x_1 + 7x_2 \leq 28$$
$$x_1, x_2 \geq 0$$
$$x_1, x_2 \in Z$$

$$\textit{Maximize } Z = 5x_1 + 6x_2$$

$$\textit{Subject to} \quad x_1 + x_2 \leq 5$$
$$4x_1 + 7x_2 \leq 28$$
$$x_1, x_2 \geq 0$$

Step 1: Remove the integrality constraints, a.k.a. Linear Relaxation

Linear Relaxation

The ***Linear Relaxation*** of an IP is the LP obtained by removing the integrality constraints!

IP

$$\begin{aligned} \textit{min}\ z &= 50x_1 + 100x_2 \\ \textit{s.t.}\quad 7x_1 + 2x_2 &\geq 28 \\ 2x_1 + 12x_2 &\geq 24 \\ x_1, x_2 &\geq 0 \\ x_1, x_2 &\in Z \end{aligned}$$

LP

$$\begin{aligned} \textit{min}\ z &= 50x_1 + 100x_2 \\ \textit{s.t.}\quad 7x_1 + 2x_2 &\geq 28 \\ 2x_1 + 12x_2 &\geq 24 \\ x_1, x_2 &\geq 0 \end{aligned}$$

Linear Relaxation

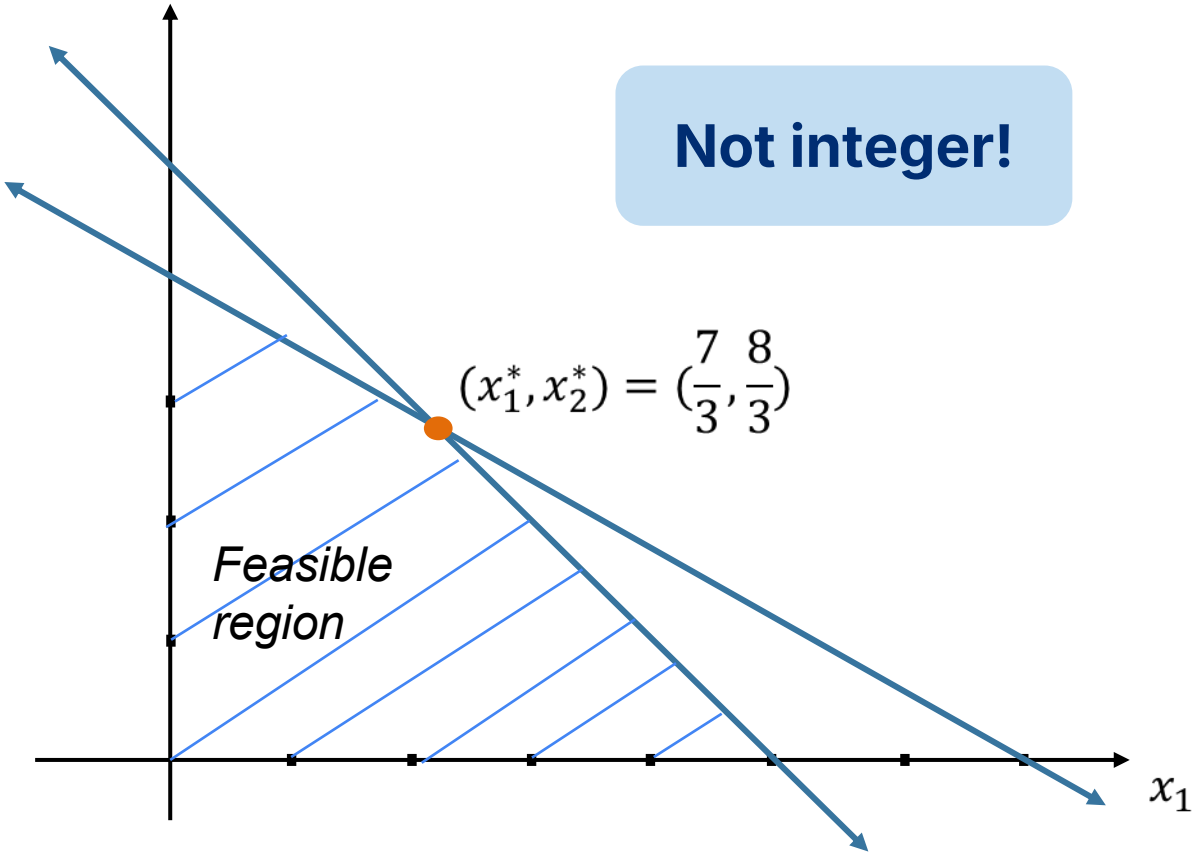
Branch and Bound Algorithm

Step 2: See whether the LP's optimal solution is integer

Maximize $Z = 5x_1 + 6x_2$

Subject to

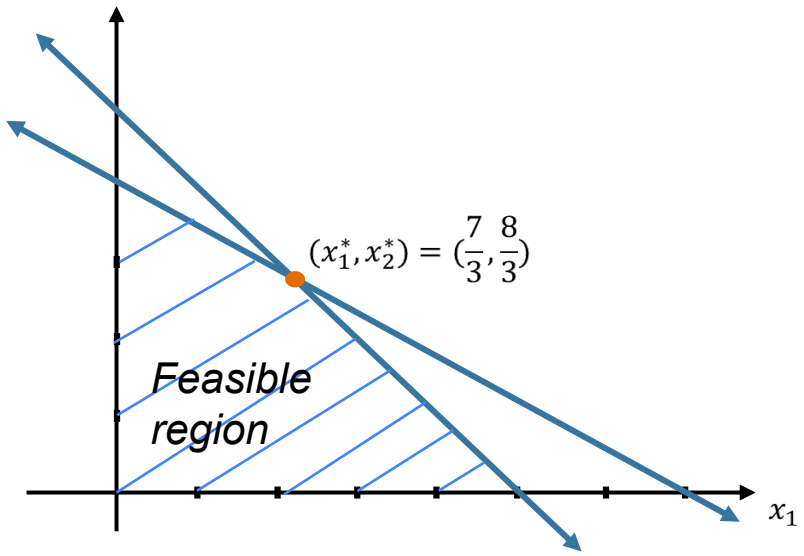
$$x_1 + x_2 \leq 5$$
$$4x_1 + 7x_2 \leq 28$$
$$x_1, x_2 \geq 0$$
$$x_1, x_2 \in Z$$

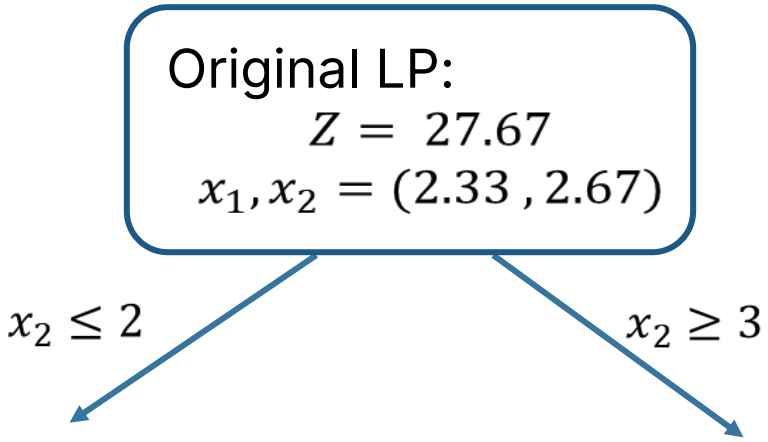


Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

Pick randomly, or the largest decimal portion, or ...





Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

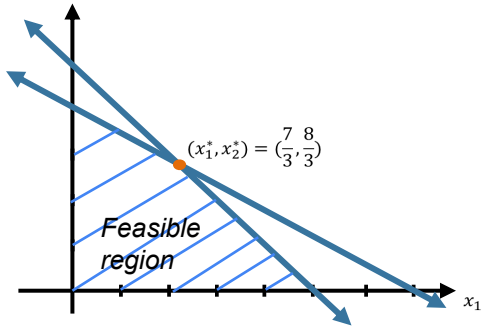
Maximize $Z = 5x_1 + 6x_2$

Subject to $x_1 + x_2 \leq 5$

$4x_1 + 7x_2 \leq 28$

$x_1, x_2 \geq 0$

$x_2 \leq 2$

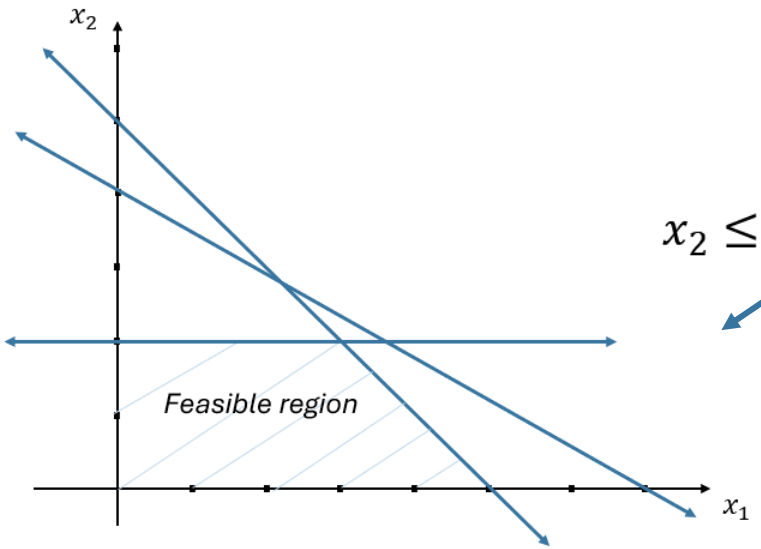


Original LP:

$Z = 27.67$

$x_1, x_2 = (2.33, 2.67)$

$x_2 \leq 2$



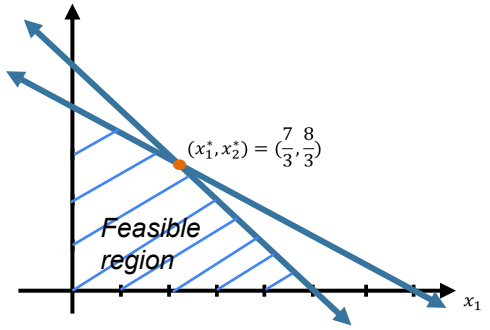
Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

Maximize $Z = 5x_1 + 6x_2$

Subject to

$$x_1 + x_2 \leq 5$$
$$4x_1 + 7x_2 \leq 28$$
$$x_1, x_2 \geq 0$$
$$x_2 \leq 2$$



Original LP:

$$Z = 27.67$$
$$x_1, x_2 = (2.33, 2.67)$$

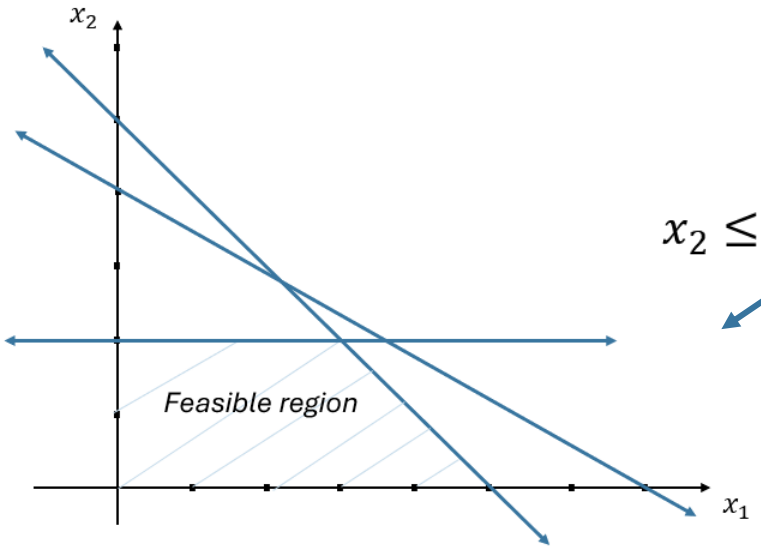
$x_2 \leq 2$

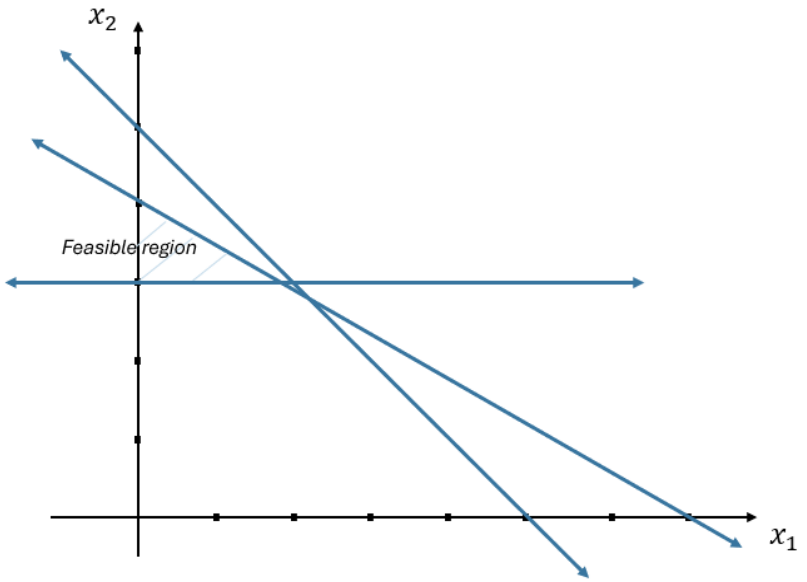
$x_2 \geq 3$

Maximize $Z = 5x_1 + 6x_2$

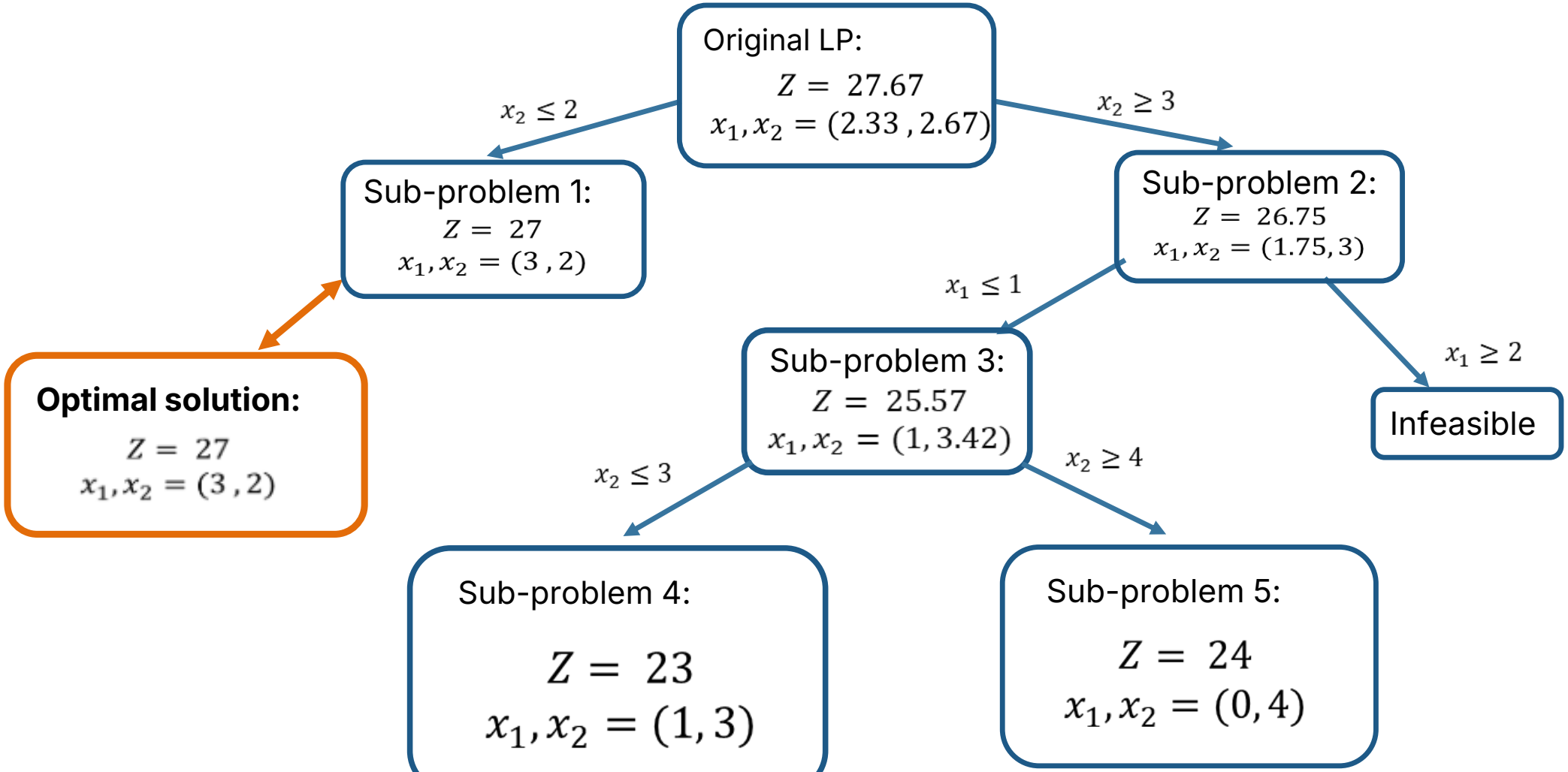
Subject to

$$x_1 + x_2 \leq 5$$
$$4x_1 + 7x_2 \leq 28$$
$$x_1, x_2 \geq 0$$
$$x_2 \geq 3$$



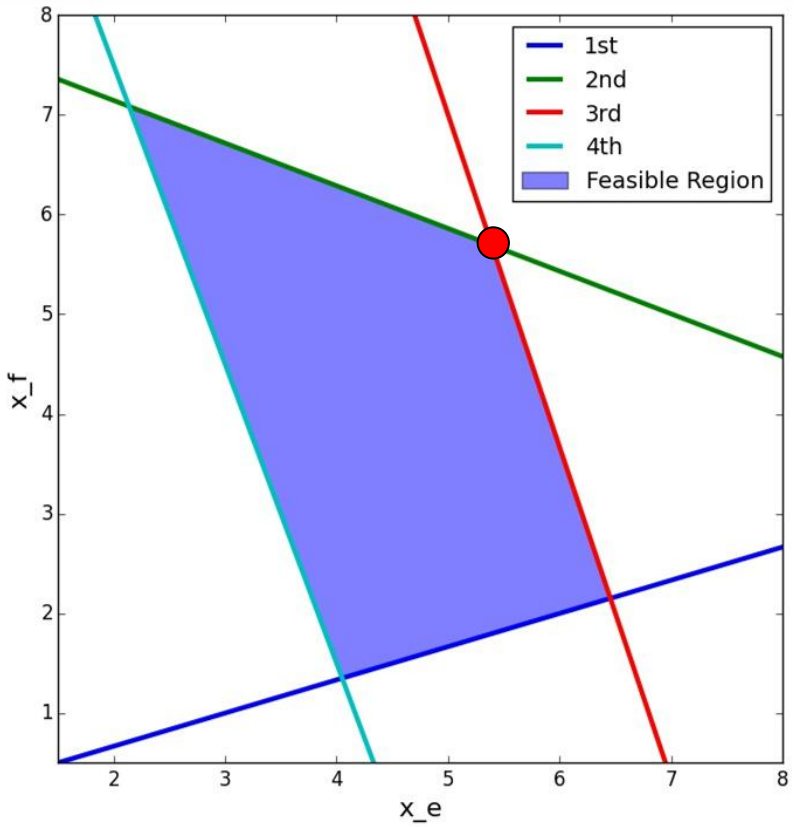


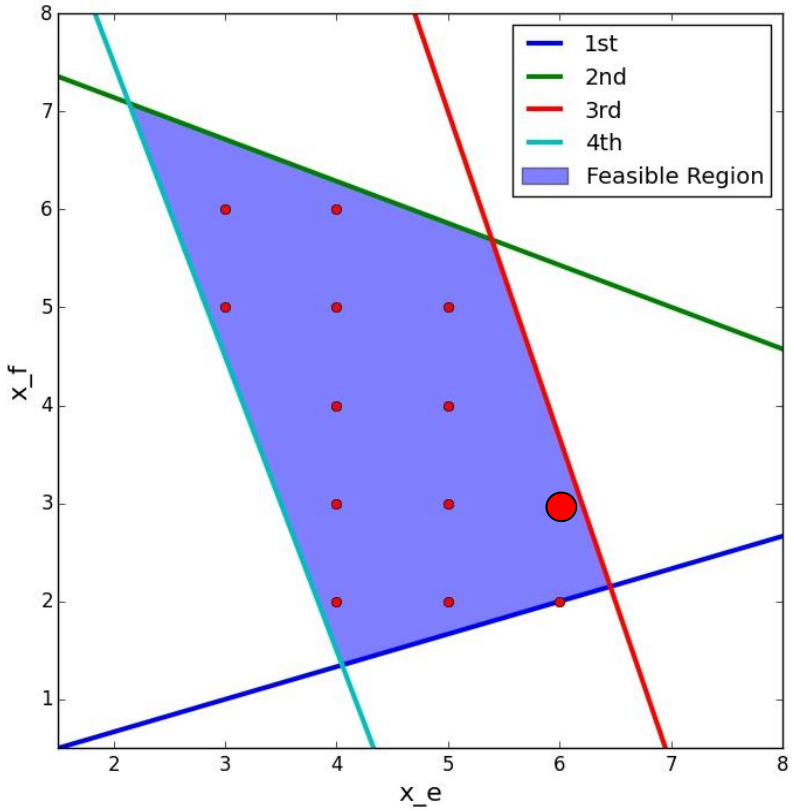
Branch and Bound Algorithm



Cutting Planes Algorithm

Cutting Planes Algorithm





We can add ***cutting planes*** that eliminate ***fractional solutions*** to the LP without eliminating any ***integer solutions***

Cutting Planes Algorithm

We can add ***cutting planes*** that eliminate ***fractional solutions*** to the LP without eliminating any ***integer solutions***

Adding the following 7 constraints, or ***cutting planes***, to the problem yields the following:

$$x_f \geq 2$$

$$x_f \leq 6$$

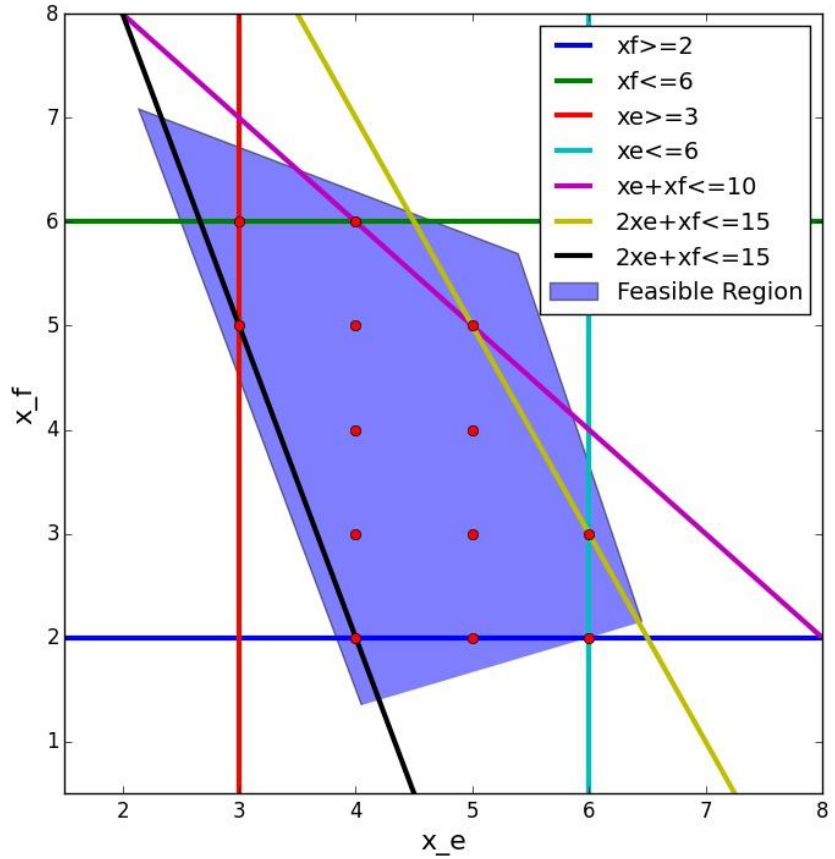
$$x_e \geq 3$$

$$x_e \leq 6$$

$$x_e + x_f \leq 10$$

$$2x_e + x_f \leq 15$$

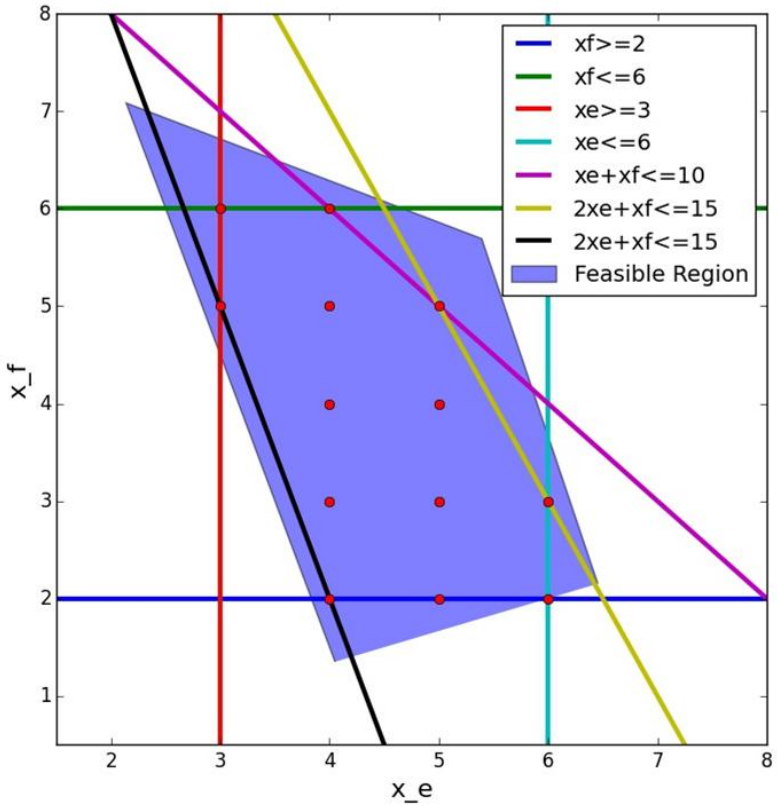
$$3x_e + x_f \geq 14$$



What's Best We Can Do?

Adding these seven ***valid inequalities*** causes every corner point of the LP to be integer

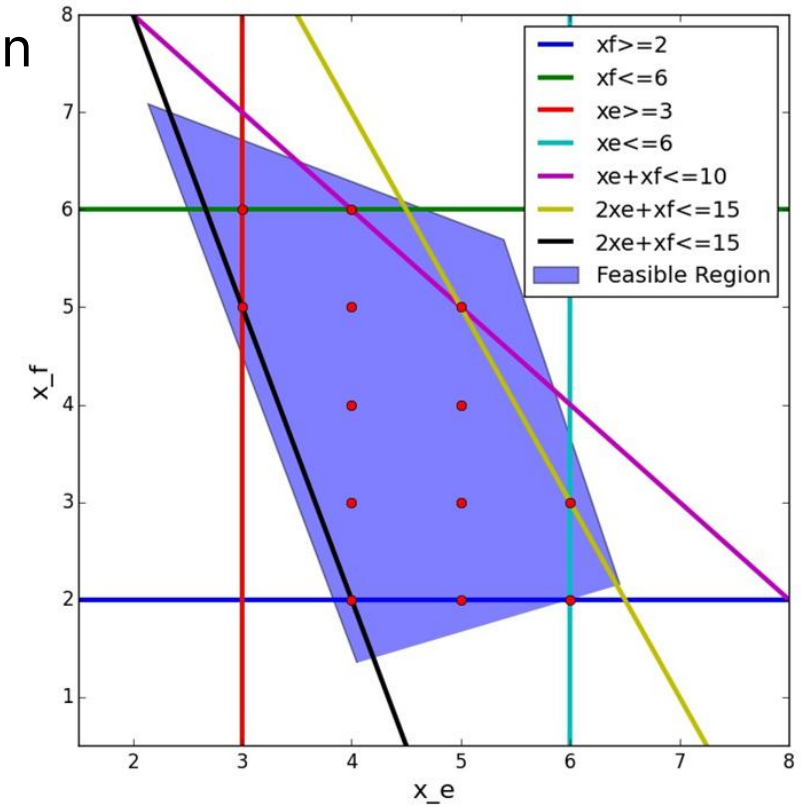
Thus, the IP can be solved by solving the LP relaxation



What's Best We Can Do?

Adding these seven ***valid inequalities*** causes every corner point of the LP to be integer

Thus, the IP can be solved by solving the LP relaxation



These 7 cutting planes form the ***convex hull of integer solutions***

- The deepest cuts are called ***facets***

How to Find Such Valid Inequalities?

There are approaches to find valid, or even facet-defining, inequalities for integer programs

- Some are problem specific
- Others are general to any integer program

These valid inequalities are typically added during the branch and bound process

- Known as ***branch and cut***

Nonlinear Programming

- Gradient & Hessian
- The Case of Equality Constraints
- The Case of Inequality Constraints
- Descent Algorithms

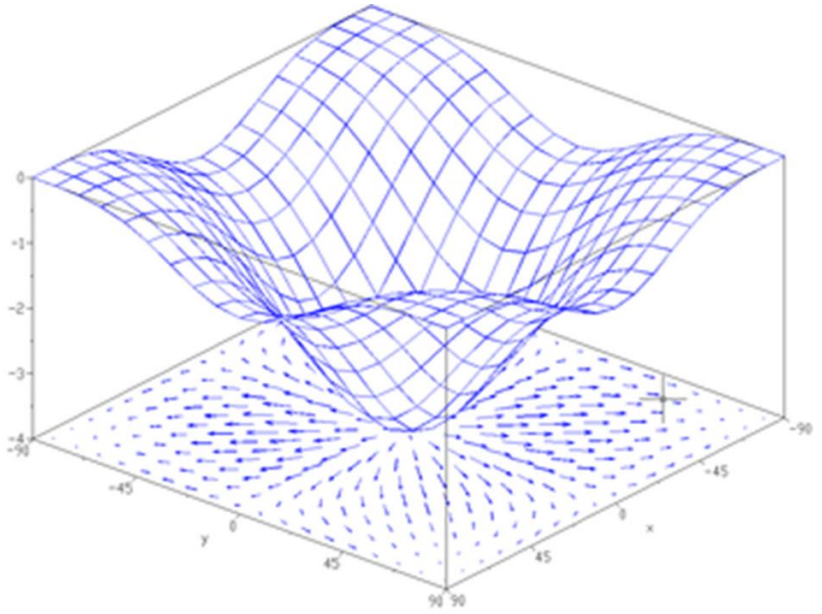
Gradient of a Function

Consider the twice-differentiable function f:

The gradient ∇f is defined as the vector of first partial derivatives: $f(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right]$$

- The gradient points in the direction of greatest increase
- Thus, $-\nabla f$ points in the direction of greatest ***decrease***
- Points where $\nabla f = 0$ are known as ***stationary points***



http://en.wikipedia.org/wiki/Gradient

Hessian Matrix of a Function

For the same twice-differentiable function $f(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$, consider representing all second partial derivatives

- First derivatives represented with gradient vector is a ∇f
- Second derivatives require a 2D matrix: carries local **curvature** information
- This is known as the **Hessian matrix H** :

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- The **Hessian matrix** is symmetric
- Off-diagonal elements mirror one another
- Can be used in verifying local (global, if convex) optimality conditions for stationary points

Optimality and the Hessian Matrix

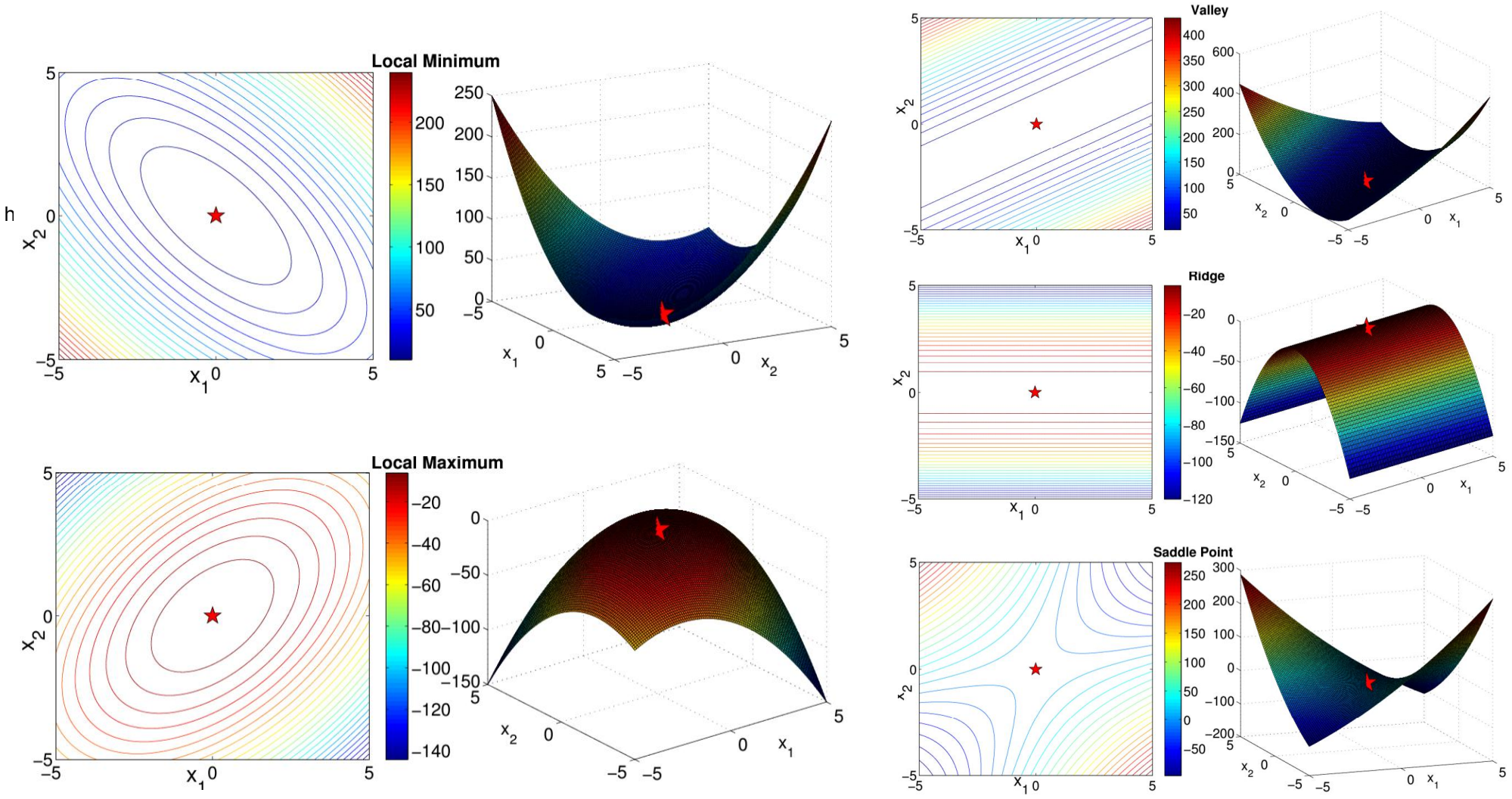
Analyze the Hessian at Critical Points:

- If the Hessian is **positive definite** at a critical point (all its eigenvalues are positive), the point is a **local minimum**.
- If it is **negative definite** (all eigenvalues are negative), it's a **local maximum.**
- If it is semi-definite (some eigenvalues are zero), the test is inconclusive, and further analysis would be helpful.

Critical point

Hessian Matrix	Nature o $x^\dagger$
positive definite	local minimizer
negative definite	local maximizer
positive semi-definite	valley
negative semi-definite	ridge
indefinite	saddle point

Optimality and the Hessian Matrix



Optimality and the Hessian Matrix

In many practical **non-quadratic** programming problems, especially those with large dimensions, direct application of the Hessian for finding the optimal solution is computationally expensive or impractical.

In such cases, we use iterative optimization algorithms like:

- Gradient descent,
- Newton's method

These methods utilize the Hessian or approximations of it to iteratively converge to an optimal solution.

Algorithmically Solving Nonlinear Optimization Problems

- Gradient Descent Method
- Newton's Method

Unconstrained Optimization

Consider a general ***unconstrained*** nonlinear optimization problem with convex, continuous, and twice differentiable objective function and convex feasible region: $\min_{x \in \mathbb{R}^n} \quad f(x)$

- In a few special cases, we can find a solution ***analytically*** using the necessary optimality condition: $\nabla f = 0$

- Typically, though, such problems must still be solved by an iterative algorithm...

Most unconstrained optimization problems have no closed form solution

General Descent Method

Starting from an initial point x^0, we want to move in an improving direction Δx

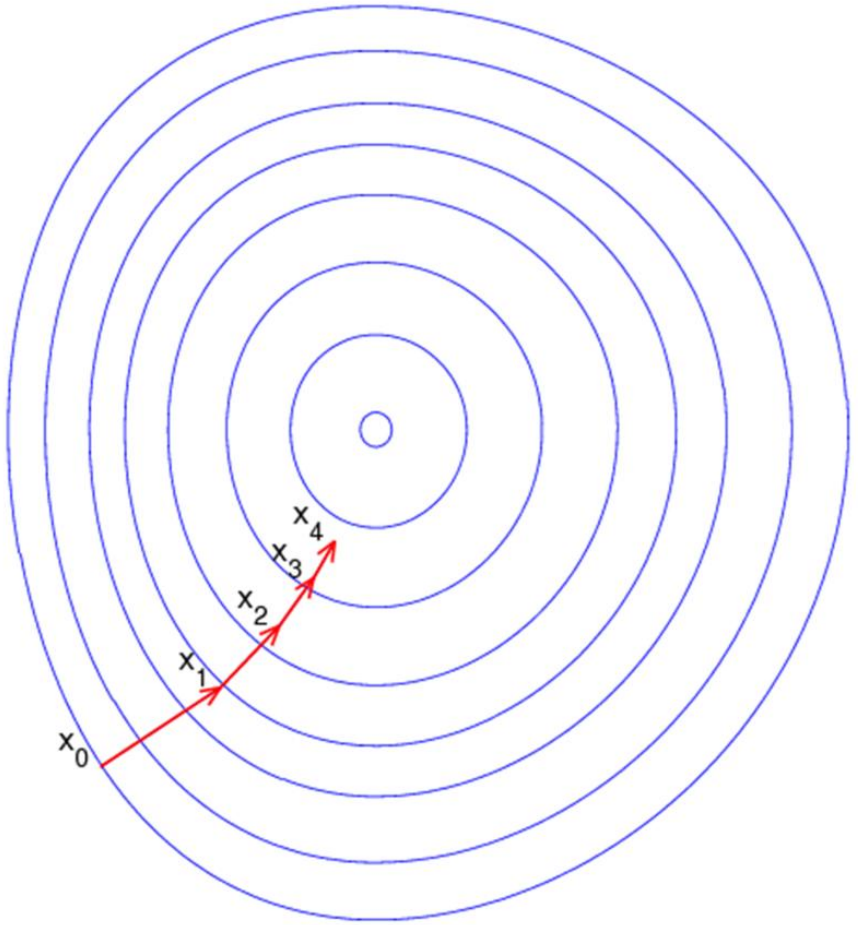
- When should we stop moving?
  - When there is (approximately) no more improving direction!
  - That is, when $\nabla f \approx 0$
  - This will lead to a stationary point
    - If the problem is a convex optimization problem, then also a ***global*** minimum

General Descent Method

Algorithm 1 General Descent Method

Input: Convex, twice continuously differentiable function $f(x)$, and starting point x^0.
Output: Optimal solution x^k.

1: Let $k \leftarrow 0$.
2: **repeat**
3: Determine descent direction Δx^k.
4: *Line search.* Choose a step size $t^k > 0$.
5: *Update.* $x^{k+1} \leftarrow x^k + t^k \Delta x^k$, and $k \leftarrow k + 1$.
6: **until** *Stopping condition satisfied.*



http://commons.wikimedia.org/wiki/File:Gradient_descent.png

Important parameters:

- Starting point x^0
- Descent direction Δx
- Step size t

Gradient Descent Method

(Steepest Descent Method)

More on Steepest Descent Method

We can use $-\nabla f$, the negative of the gradient, for the ***best*** improving direction for a minimization problem

- This is known as the ***gradient descent method***
- However, it only improves so far, before it needs updated
  - The gradient at point x^{k+1} is not likely the same as at point x^k
- The step size t can be chosen using one of several ***line search*** approaches

Newton's Method

(Deflected Gradient Descent Method)

Using Additional Information

- The Steepest (Gradient) Descent Method uses only gradient ∇f information
- More information is available from the Hessian H, which provides information on the ***curvature*** of the function $f(x)$
- The step direction $-H^{-1}\nabla f$ (rather than just the negative of the gradient $-\nabla f$) is called the ***deflected gradient (Newton's Method)***
  - It has a fixed step size $t = 1$
  - Incorporates second-derivative information

Newton's Method

The following algorithm is a generic version of Newton's Method

Algorithm 2 Newton's Method

Input: Convex, twice continuously differentiable function $f(x)$, and starting point x^0.
Output: Optimal solution x^k.

1: Let $k \leftarrow 0$, and set $t \leftarrow 1$.
2: **repeat**
3: Set $\Delta x^k \leftarrow -H^{-1}(x^k)\nabla f(x^k)$.
4: *Update.* $x^{k+1} \leftarrow x^k + t\Delta x^k$, and $k \leftarrow k + 1$.
5: **until** *Stopping condition satisfied.*

Optimality Conditions: Equality Case

Consider the equality-constrained nonlinear program:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & g_i(x) = 0, \ i = 1, \ldots, m. \end{aligned}$$

- Objective is, in general, some nonlinear, differentiable function
- Optimizing over the real variables (i.e., unrestricted)
- There are m potentially nonlinear constraints

We use the following function, known as the ***Lagrangian***:

$$L(x, \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x)$$

Optimality Conditions: Equality Case

Consider the equality-constrained nonlinear program:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & g_i(x) = 0, \ i = 1, \ldots, m. \end{aligned}$$

- Objective is, in general, some nonlinear, differentiable function
- Optimizing over the real variables (i.e., unrestricted)
- There are m potentially nonlinear constraints

We use the following function, known as the ***Lagrangian***:

$$L(x, \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x)$$

The condition $\nabla L = 0$ is a ***necessary*** condition for optimality

Optimality Conditions: Equality Case

Again, the Lagrangian function is: $L(x, \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x)$

- The condition $\nabla L = \mathbf{0}$ implies: $\frac{\partial L}{\partial x} = \nabla f(x) + \sum_{i=1}^{m} \lambda_i \nabla g_i(x) = 0$

and $\frac{\partial L}{\partial \lambda_i} = g_i(x) = 0, \;\; i = 1, \ldots, m$

Optimality Conditions: Equality Case

Again, the Lagrangian function is: $L(x, \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x)$

- The condition $\nabla L = \mathbf{0}$ implies: $\frac{\partial L}{\partial x} = \nabla f(x) + \sum_{i=1}^{m} \lambda_i \nabla g_i(x) = 0$

and $\frac{\partial L}{\partial \lambda_i} = g_i(x) = 0, \;\; i = 1, \ldots, m$

The first condition states that the gradient of the objective can be represented as a linear combination of the constraint gradients

These conditions jointly represent ***necessary optimality conditions*** for equality constrained nonlinear optimization problems!

Optimality Conditions: Equality Case

Again, the Lagrangian function is: $L(x, \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x)$

- The condition $\nabla L = \mathbf{0}$ implies: $\frac{\partial L}{\partial x} = \nabla f(x) + \sum_{i=1}^{m} \lambda_i \nabla g_i(x) = 0$

and $\frac{\partial L}{\partial \lambda_i} = g_i(x) = 0, \; i = 1, \ldots, m$

The first condition states that the gradient of the objective can be represented as a linear combination of the constraint gradients

These conditions jointly represent ***necessary optimality conditions*** for equality constrained nonlinear optimization problems!

→ Any point x satisfying these conditions is a ***stationary point*** for the Lagrangian function

Optimality Conditions: Inequality Case

Today we consider the more general **inequality-constrained NLP**:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \leq 0, \ i = 1, \ldots, m \\ & h_\ell(x) = 0, \ \ell = 1, \ldots, p. \end{aligned}$$

- Similar to the ***equality*** constrained case, but with new ***inequalities*** present
- The corresponding Lagrangian function is simply:

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{\ell=1}^{p} \nu_\ell h_\ell(x)$$

- The Lagrange multiplier associated with each $g_i(x) \leq 0 : \lambda_i$
- The Lagrange multiplier associated with each $h_\ell(x) = 0 : \nu_\ell$

Karush-Kuhn-Tucker (KKT) Conditions

Represent ***necessary optimality conditions*** for any solution x to ***inequality***-constrained nonlinear optimization problems:

1. $g_i(x) \leq 0, i = 1, \ldots, m; \; h_\ell(x) = 0, \ell = 1, \ldots, p$ ***1) Primal feasible***

The solution must satisfy all the constraints of the problem.

Karush-Kuhn-Tucker (KKT) Conditions

Represent ***necessary optimality conditions*** for any solution x to ***inequality***-constrained nonlinear optimization problems:

1. $g_i(x) \leq 0, i = 1, \ldots, m;\ h_\ell(x) = 0, \ell = 1, \ldots, p$
2. $\lambda_i \geq 0, i = 1, \ldots, m$

1) Primal feasible

2) Sign restrictions

Karush-Kuhn-Tucker (KKT) Conditions

Represent ***necessary optimality conditions*** for any solution x to ***inequality***-constrained nonlinear optimization problems:

1. $g_i(x) \leq 0, i = 1, \ldots, m; \; h_\ell(x) = 0, \ell = 1, \ldots, p$
2. $\lambda_i \geq 0, i = 1, \ldots, m$
3. $\lambda_i g_i(x) = 0, i = 1, \ldots, m$

1) Primal feasible
2) Sign restrictions
3) Complementary Slackness

For each inequality constraint, either the constraint is 'active' (i.e., it holds as an equality at the optimal point), or its corresponding Lagrange multiplier is zero

Karush-Kuhn-Tucker (KKT) Conditions

Represent ***necessary optimality conditions*** for any solution x to ***inequality***-constrained nonlinear optimization problems:

1. $g_i(x) \leq 0, i = 1, \ldots, m;\ h_\ell(x) = 0, \ell = 1, \ldots, p$
2. $\lambda_i \geq 0, i = 1, \ldots, m$
3. $\lambda_i g_i(x) = 0, i = 1, \ldots, m$

1) Primal feasible
2) Sign restrictions
3) Complementary Slackness

For each inequality constraint, either the constraint is 'active' (i.e., it holds as an equality at the optimal point), or its corresponding Lagrange multiplier is zero

If λ_i is the multiplier for the i-th constraint $g_i(x) \leq 0$, then $\lambda_i g_i(x) = 0$ at the optimal solution.

This means that if $g_i(x) < 0$ (constraint not 'active'), then $\lambda_i = 0$, and if $\lambda_i > 0$, then $g_i(x) = 0$ (constraint is 'active')

Karush-Kuhn-Tucker (KKT) Conditions

Represent ***necessary optimality conditions*** for any solution x to ***inequality***-constrained nonlinear optimization problems:

1. $g_i(x) \leq 0, i = 1, \ldots, m; \; h_\ell(x) = 0, \ell = 1, \ldots, p$
2. $\lambda_i \geq 0, i = 1, \ldots, m$
3. $\lambda_i g_i(x) = 0, i = 1, \ldots, m$
4. $\nabla f(x) + \sum_{i=1}^{m} \lambda_i \nabla g_i(x) + \sum_{\ell=1}^{p} \nu_\ell \nabla h_\ell(x) = 0$

1) Primal feasible
2) Sign restrictions
3) Complementary Slackness
4) Stationarity

The last condition states that the gradient of the objective can be represented as a linear combination of the constraint gradients

A Note on Sign Restrictions

The sign restrictions on λ_i vary based on a **minimization** or **maximization** problem

The interpretation of λ_i is: ***what happens to the objective function when the right-hand side of inequality constraint i increases***

There are four cases, depending on ***objective function direction*** and ***sense of the inequality constraint***:

1. min $f(x) : g_i(x) \leq 0 \Rightarrow \lambda_i \geq 0$
2. min $f(x) : g_i(x) \geq 0 \Rightarrow \lambda_i \leq 0$
3. max $f(x) : g_i(x) \geq 0 \Rightarrow \lambda_i \geq 0$
4. max $f(x) : g_i(x) \leq 0 \Rightarrow \lambda_i \leq 0$